The Design and Development of Animators Hive Hub, a Web Platform for Animators

Edem Angel Edem Hogan (P0013366)

AfriHUB

Abstract

Animators Hive Hub is a web platform for animators to share their work and connect with other animators, people in the industry, and enthusiasts. It is open to anyone. Animators currently post their work on YouTube, Instagram, Twitter, and other general platforms. Their work competes for attention with many other kinds of content. A platform dedicated to animation gives people who want to watch animation one place to find it. The project involved development, testing, and live deployment. Users can upload videos, create playlists and add videos to them, sign in with Google, comment on videos, upvote videos, create and join communities, and sort videos by newest or most upvoted.

The app was built with SvelteKit and TypeScript and deployed to Cloudflare Workers. It stores application records in D1 and uploaded media in R2. The initial design used React and Supabase, but the implementation changed during development. The finished platform provides the main features for sharing and discovering animations.

Contents

Abstract 2
1 Introduction 4
The system developed 4
Justification for the method or framework used 4
The solution that emerged 4
Aims 5
Objectives 5
Overview of the remaining chapters 6
2 Analysis 7
Introduction 7
Requirements 7
Functional requirements 7
System-wide non-functional requirements 8
Use cases 8
Upload and publish an animation 8
Find and respond to an animation 9
Create a playlist 9
Create an account and sign in 9
Create and join a community 10
Architecture 10
Interfaces with other systems 10
Technical architectures for development and implementation 10
Initial class diagram 11
Conclusion 11
3 Design 12
Introduction 12
Structural model 12
Behavioural model 13
Conclusion 14
4 Implementation 15
Introduction 15
Choice of programming language 15
Data storage and upload 16
Authentication and access 17
System cutover 17
Data migration 17
Training 17
Conclusion 17
5 Other project issues 19
Project management 19
Risk management 20
Configuration management 21
Testing 21
6 Conclusion 23
Evaluation of aims and objectives 23
Future development 24
References 25
Appendices 26
A Test scripts 26
B User guide 27
C Source code 27

Figures

Figure 1: Initial class diagram 11

Figure 2: Current class diagram 12

Figure 3: Animation upload sequence 14

Tables

Table 1: Functional requirements 8

Table 2: Project work schedule 19

Table 3: Project risks and actions 21

Table 4: Test results 22

Table 5: Test scripts 26

1 Introduction

The system developed

This is a web app that lets users upload animations, join communities, comment on other people's animations, add animations to playlists, and connect with other animators in communities.

Justification for the method or framework used

I decided to host the app on Cloudflare. I wanted the database and files to also be on Cloudflare to reduce response latency. The initial design used a React front end and Supabase for the back end. I changed the front end to SvelteKit for performance purposes. The server code is in the same codebase. The final version stores account and animation data in Cloudflare D1 and uploaded files in Cloudflare R2. The early design included similarity search, but that feature was removed.

Keeping the browser interface and server routes in one SvelteKit project meant the page and the code that supplies its data could be developed together. The app uses server-side loading for the main pages. This lets the server obtain the user's session and page data before it renders the response. The API routes keep the rules for writes on the server, where a visitor cannot change them by editing browser code.

D1 was selected for information such as users, animations, comments, votes, and playlists because these records refer to one another. The database can enforce those references and keep counts with their related records. R2 was selected for video and image files because they are files rather than rows of application data. The first Cloudflare design used Vectorize and an embedding model for similarity search. The application did not query those embeddings, so the final implementation removed both services. The finished app uses ordinary search and feed filters instead.

The solution that emerged

No hardware was developed for this project. The software that was created is a SvelteKit web app.

The result is a website that works in a browser. The browser displays pages and sends requests to the server. The server reads and writes application data in D1 and handles media in R2. The same project contains both the interface and the server code, so a feature such as uploading an animation has a page, a server route, a database operation, and a media operation in one codebase.

Aims

The aim is to build a dedicated place where animators can come, share their work, find other animators' work, connect with other animators and enthusiasts, and easily have their work discovered by the world.

The project focuses on a specific problem: general video platforms contain many kinds of content, which makes it harder for a viewer who wants animation to find it. A dedicated feed gives animation its own place. The project does not claim that a specialised feed guarantees that every animator will be discovered. It provides a focused way to browse and share the work.

Objectives

- The main objective is to have a feed where users can come and see animators' posts, sorted by the top post or the newest post. From that feed, they can click on a video and see its comments, click on the animator's profile and see more videos from the animator, and click on the video's playlist to see other videos the animator added to that playlist. That is expected to facilitate easy discovery of animators' work.

The feed is the starting point for discovery. It provides a new order and a top order based on upvotes. A visitor can search for animations and browse by community or creator. Each animation has its own page, where the visitor can view the video and its comments. A signed-in user can respond with an upvote or comment. A public profile provides another route to an animator's work, and a playlist groups animations selected by its owner.

The account and ownership objectives support these activities. A user must be able to create an account and return to it later. The server must know which user is signed in before allowing an upload or a change to private account data. It must also know who owns an animation, comment, playlist, or account before allowing an edit or deletion. This protects each user's work from changes by another account.

Overview of the remaining chapters

Chapter 2 presents the system requirements, use cases, and architecture. Chapter 3 describes the system's structural and behavioural design. Chapter 4 explains the implementation, including the programming language, system cutover, data migration, and training. Chapter 5 covers project management, risk management, configuration management, and testing. Chapter 6 evaluates the project aims and objectives and discusses future development.

2 Analysis

Introduction

This chapter analyses the system requirements, use cases, and architecture for Animators Hive Hub.

Requirements

The system is intended for animators who want to show their work and viewers who want to find animation. Anyone can browse the public feed and public profiles. A signed-in user can contribute to the platform and manage their own account and content. The first version concentrates on sharing and discovery rather than employment, private messaging, or payments.

Functional requirements

ID	Requirement	Implementation
FR1	A user can create an account and sign in.	Email and password accounts and Google sign-in are available.
FR2	A visitor can browse animations.	The discover feed and animation pages are public.
FR3	A signed-in user can upload an animation with a title, description, and thumbnail.	The media is uploaded to R2 and the animation record is saved in D1.
FR4	A user can find animations by feed order, community, creator, or text search.	The feed supports new and top sorting, filtering, and search.
FR5	A signed-in user can comment on or upvote an animation.	The animation page displays comments and the current upvote count.
FR6	Users can create communities and join or leave them.	Community pages show animations and membership state.
FR7	A signed-in user can create playlists and	Playlist pages show their saved animations.

ID	Requirement	Implementation
	add animations to them.	
FR8	A user can view a public profile.	Public profiles show the user's information and animations.
FR9	Owners can update or remove their own content.	Server routes check ownership before changing animations, comments, playlists, or accounts.

Table 1: Functional requirements.

The first design also included community messages, similarity search, and a job board. These features are not in the finished system. The community feature supports membership and animation discovery, but it does not include a chat.

System-wide non-functional requirements

- Only signed-in users can create or change account-owned content.
- Users can change or delete only content they own.
- User input must be checked on the server before it is stored.
- Uploaded videos and images must have supported media types and remain within their size limits.
- The site should work on phone-sized screens as well as desktop screens.
- Public pages should be available to visitors without requiring an account.

Use cases

Upload and publish an animation

Actor: signed-in user. **Precondition:** the user has an account and a video file within the upload limit.

The user opens the upload form, enters a title and optional description, and selects a video. The browser creates a thumbnail from the video. It sends the video and thumbnail to the media endpoint. The server checks the file type and size, stores each file in R2, and returns a media path.

The user submits the animation details. The server checks the title, description, media paths, and optional community, then creates the animation record in D1. The animation appears in the feed and has a page of its own.

If the user is not signed in, the upload request is refused. If a file is too large or has an unsupported type, the server returns an error and does not create an animation record. If the selected community does not exist, the animation cannot be created.

Find and respond to an animation

Actor: visitor or signed-in user. **Precondition:** an animation has been published.

The visitor opens the discover page and chooses new or top order, searches for a title, or opens a community or public profile. The visitor selects an animation to open its detail page. A signed-in user can upvote it or add a comment. The server verifies the session and updates the related records and counts in D1. A user can remove their own comment; another user cannot remove it.

Create a playlist

Actor: signed-in user. **Precondition:** the user has an account.

The user creates a playlist, opens an animation, and adds it to the playlist. The server checks that the playlist belongs to the user and that the animation exists. The user can later rename the playlist, remove an item, or delete the playlist. Other users can view public playlist content, but cannot change the owner's list.

Create an account and sign in

Actor: visitor. **Precondition:** the visitor has an email address, or a Google account with a verified email address.

The visitor chooses email sign-up or Google sign-in. For email sign-up, the visitor enters a username, email address, and password. The server validates the values and creates the user record. For Google sign-in, the application redirects the visitor to Google and receives the response at its callback route. The application checks the response and the verified email before it creates or finds the matching account. In either case, the application creates a session and takes the user to the signed-in part of the site.

If an email or username is already in use, the server refuses the new account. If Google does not return a verified email, the application refuses the sign-in. The system does not send email, so it does not provide password reset or email verification for password accounts.

Create and join a community

Actor: signed-in user. **Precondition:** the user has an account.

The user creates a community by supplying its name and optional description. The system creates a community page and records its creator as its first member. Another user can open the page and join. A member can leave later. The membership table prevents the same user being counted more than once. The community page can show animations linked to that community, but it does not provide a message board or chat.

Architecture

The system has a browser client, a SvelteKit application running on Cloudflare Workers, a D1 database, an R2 bucket, and Google's OAuth service. A request for a page or API route reaches the Worker. The server reads or changes D1 records, or stores and retrieves media in R2. The browser receives page data or a media path. Media is served through the application's media route, which supports partial responses so the browser can seek within a video. Cloudflare describes Workers as a platform for running application code and D1 as a SQL database with SQLite semantics (Cloudflare, 2026a; Cloudflare, 2026b). R2 stores objects, which suit the video and image files in this project (Cloudflare, 2026c).

Interfaces with other systems

Users can sign in with Google. The app uses Google OAuth to start the sign-in process and obtain the user's verified account details. The app is hosted on Cloudflare Workers and uses Cloudflare D1 and R2.

Technical architectures for development and implementation

After the user uploads a video, the video is stored in Cloudflare R2. Account, animation, community, comment, upvote, playlist, and playlist-item records are stored in Cloudflare D1. The final design does not use Vectorize or embeddings.

The D1 database is relational. Users own animations and playlists. Animations can be linked to a community and can have comments and upvotes. A playlist item connects one animation to one playlist. Foreign keys define those relationships, and indexes support the common feed orders and lookups by creator and community. Community membership is held in a separate membership table. The application uses the database to calculate comment, vote, and member counts, so those values reflect the saved records.

The application has separate routes for pages and operations. Page routes load the feed, profile, community, playlist, or animation data that the page needs. API routes accept changes such as a new animation, comment, vote, or playlist item. The server checks the request and the user's session before writing. The database access functions are kept in one module so that the same ownership and data rules apply wherever an operation is used.

Initial class diagram

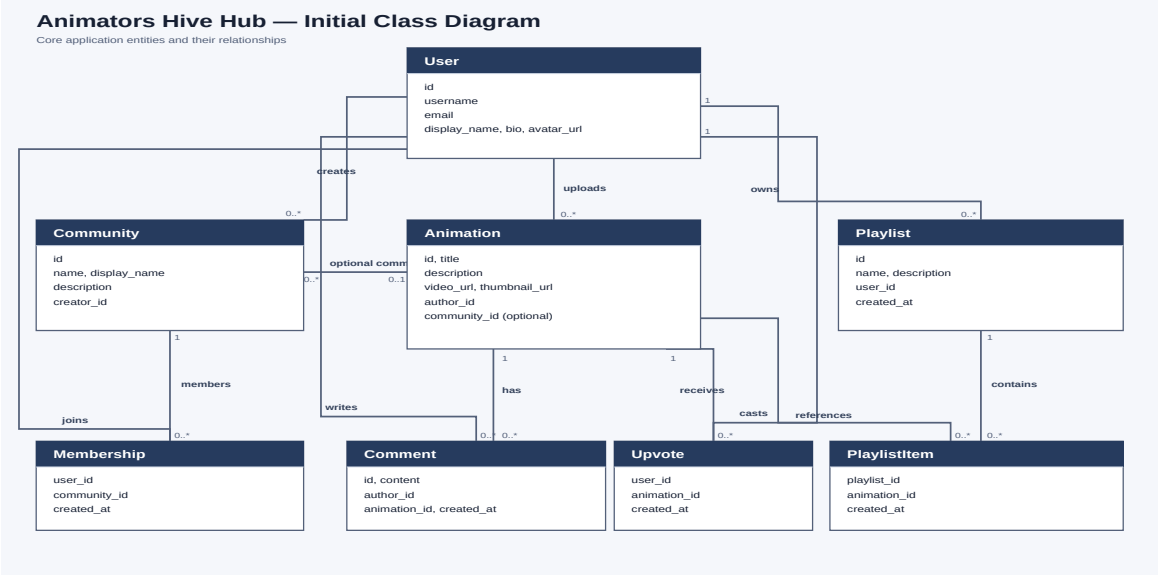


Figure 1: Initial class diagram.

Conclusion

This chapter defined the requirements, use case, and technical architecture. The final system stores uploaded videos in R2 and application records in D1. These requirements informed the design stage.

3 Design

Introduction

This chapter describes the design of Animators Hive Hub. It covers the structural model, which sets out the entities and their relationships, and the behavioural model, which sets out how the system responds to user actions.

Structural model

The entities and relationships followed from the idea and the functional requirements. The main entities are users, animations, playlists, communities, comments, and upvotes.

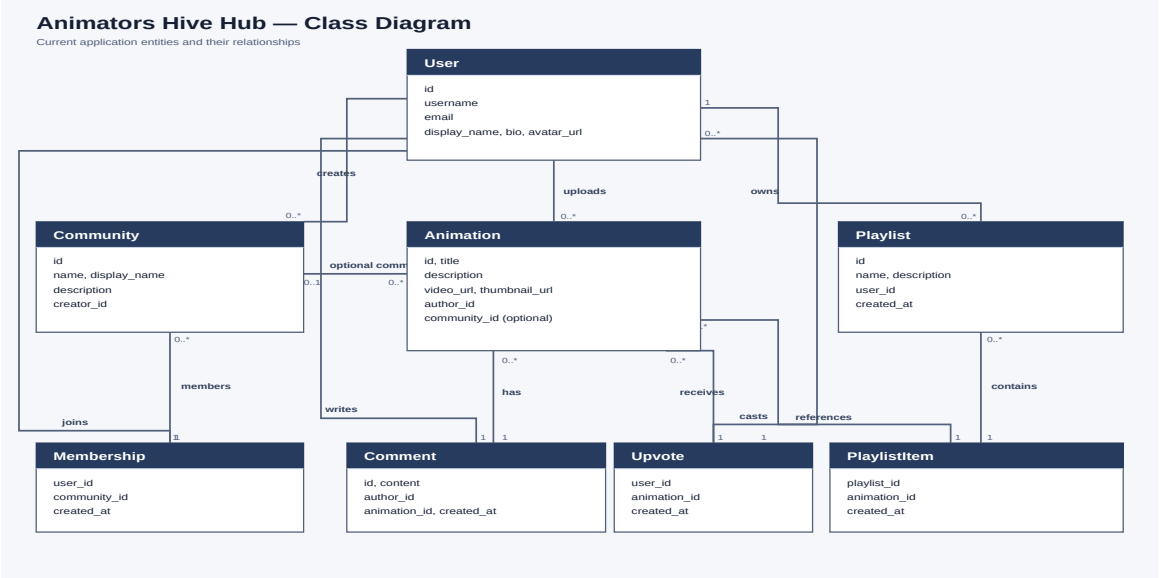


Figure 2: Current class diagram.

An animation belongs to one user and may belong to a community. A comment belongs to an animation and its author. An upvote connects a user to an animation, with one upvote for each user-animation pair. A playlist belongs to one user and contains playlist items that refer to animations. These relationships allow the application to show counts and enforce ownership.

The user record stores the account email, username, password hash when there is one, and profile fields such as display name, biography, avatar, location, and website. An animation record stores its title, description, media paths, author, optional community, creation time, and upvote and comment counts. A community record stores its name and creator; community members are kept

separately so that joining and leaving can be represented. A comment stores its text, author, animation, and creation time. A playlist item links one playlist to one animation, and the database prevents duplicate items in the same playlist.

This model follows the actions available in the interface. For example, a vote is a relationship between a user and an animation rather than a number held only in the browser. The stored relationship lets the application determine whether the current user has already voted. Likewise, a playlist item is its own record, so the user can remove one item without deleting the animation itself.

Behavioural model

The upload sequence shows a signed-in user selecting a video, the browser sending it to the application, and the application storing the media in R2 before saving the animation record in D1. Other users can then load the animation and add comments.

Only signed-in users can comment on an animation. Only the original poster of a comment can delete it. The same ownership idea applies to playlists, animations, and user accounts: each write is checked by the server against the signed-in user's identity.

The upload sequence has two writes with different purposes. First, the file bytes are sent to R2 and the application returns a path. Then the animation details and the returned path are written to D1. If the database write fails after an upload, the media object may remain without an animation record. A future improvement could clean up such unused objects. The current sequence was chosen so the relational record never points to a media path that the application did not issue.

The page flows also distinguish visitors from signed-in users. Anyone can browse an animation or profile. A visitor who selects an action that changes data is asked to sign in. Once signed in, the application can perform the action and return updated page data. This keeps public discovery available while protecting write operations.

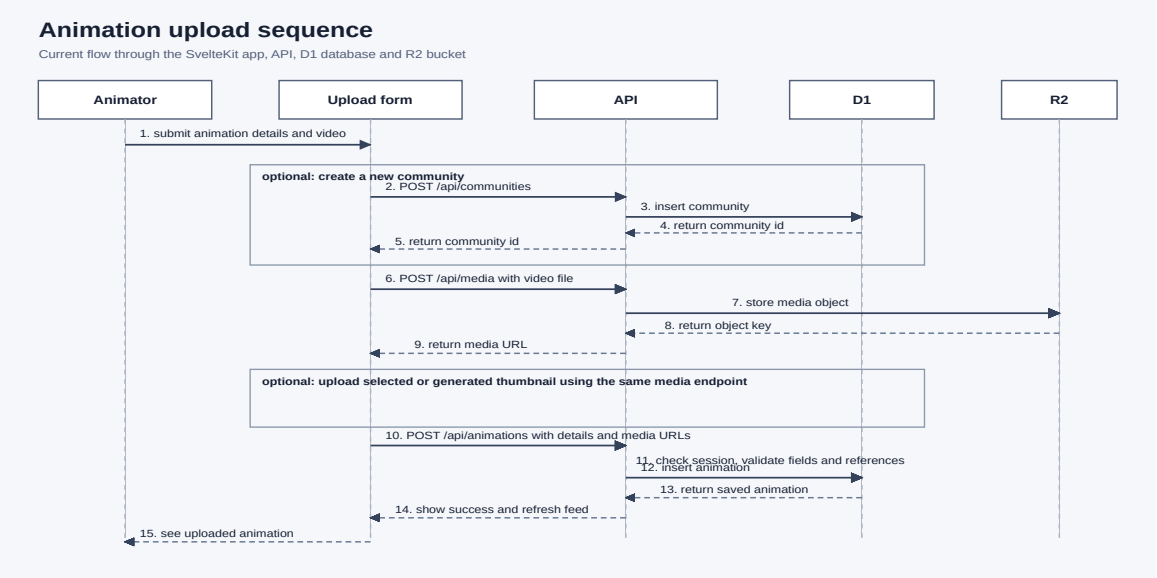


Figure 3: Animation upload sequence.

Conclusion

The design uses users, animations, communities, comments, upvotes, playlists, and playlist items as its main entities. The class diagram shows their relationships. The sequence diagram shows how an uploaded animation becomes available for other users to view and comment on.

The design keeps the media file separate from the animation record. The R2 object holds the uploaded bytes. The D1 animation row holds the title, description, owner, optional community, and paths to its video and thumbnail. This avoids storing large media inside the relational database and lets the page retrieve the record and the media through the routes designed for each purpose.

4 Implementation

Introduction

This chapter describes how the design was implemented. The application uses TypeScript, SvelteKit, and Svelte. It runs on Cloudflare Workers, stores records in D1, and stores uploaded media in R2.

Choice of programming language

I used TypeScript with SvelteKit. SvelteKit provides the pages and server routes in one project. Its route and server load files let the server load page data before sending the page to the browser (Svelte, 2026). TypeScript is used for both the browser code and server code. Tailwind CSS is used for the interface.

The server routes handle requests such as sign-in, animation creation, comments, upvotes, community membership, and playlists. Database queries are kept in one server module. This keeps the route handlers focused on receiving requests, checking access, and returning responses.

The app uses SvelteKit server rendering for the main pages. The server reads the session and supplies the initial page data. This avoids making the first page view depend on a second browser request for its main content. The same server code is used during local development and in the deployed Worker, with local bindings for development and Cloudflare bindings in production.

The server-rendered pages include the discover feed, animation detail, community, playlist, and profile pages. A user who already has a session can receive the signed-in view with the first response. This also gives the server a consistent place to decide whether a page is public or requires a signed-in user. The browser still handles interactions such as opening dialogs, selecting files, and changing the feed order.

Data storage and upload

Animation details and user information are stored in D1. The database schema is applied through SQL migrations. Uploaded video and image files are stored in R2. A signed-in user uploads a video or image first; the server stores it in R2 and returns a media path. The animation record then stores that path with its title, description, and optional community. Video uploads are limited to 50 MB and image uploads to 5 MB.

Each uploaded file is stored under a randomly generated identifier. The server accepts video and image media types and checks the actual request body size as well as the declared content length. The animation creation route accepts only a media path generated by the application. This prevents a user from attaching an arbitrary external URL as the animation's video or thumbnail. The media route reads the matching object from R2 and supports byte-range requests used by video playback.

The database schema uses migrations so the tables can be created consistently in local development and production. The tables represent users, communities, animations, upvotes, comments, playlists, playlist items, sessions, and community members. Timestamps are stored as integer milliseconds. Identifiers are generated as UUIDs. Foreign keys define how dependent data behaves when a user, animation, or playlist is removed. Indexes support feed order and common filters.

The first migration creates the main application tables. Later migrations add sessions and community membership. This lets the application evolve without replacing the complete schema by hand. Local development uses a local D1 database, so testing an upload or account change does not write into the production database. Production uses the database binding declared in the Worker configuration.

The animation, comment, and upvote counts are stored with the animation row. When a comment or upvote changes, the database operation updates the related count from the saved rows rather than relying on a count held only in the browser. This keeps the values consistent if more than one request changes the same animation.

Authentication and access

Users can create an account with an email address and password or sign in with Google. The Google sign-in route uses the web-server OAuth flow, then checks that the returned email is verified before creating or finding the account (Google, 2026). The server checks the session on requests that require an account. Uploading, commenting, upvoting, and changing a profile require sign-in. The server also checks ownership before users change or remove their content.

For password accounts, the server stores a salted PBKDF2 hash rather than the submitted password. A session uses a random token in an HTTP-only cookie; the database stores a hash of that token and its expiry. The cookie is marked secure on HTTPS and uses the same-site lax setting. Logging out deletes the session record. Changing a password ends the user's other sessions. The server also applies request limits to authentication, uploads, and other writes.

System cutover

The project began as a React application using Supabase. During development, I changed it to SvelteKit and Cloudflare. This was a change to the project implementation, not a replacement of an operational system used by existing users.

Data migration

No real user data was moved from Supabase into the finished system. The old Supabase project was no longer available when the new version was completed.

Training

No user training or user guide was provided.

Conclusion

The final implementation is a SvelteKit application deployed on Cloudflare Workers. D1 stores application records and R2 stores uploaded media. This replaced the earlier React and Supabase design. No existing system or user data was moved into the finished application.

The repository is connected to a Workers Builds pipeline. A push to the project branch starts a build and deploys the application. The project configuration supplies the D1 and R2 bindings, while secrets such as the session secret and Google OAuth credentials are configured for the production Worker. The local development server uses a separate local database and local secret values.

The move also changed how the application was rendered. The earlier project was a React single-page app. The final version uses SvelteKit server rendering for its main pages. This required the server to load data and authentication state before returning the page. It also meant that the Worker had to run the SvelteKit server build instead of serving only static files. The deployment failure exposed configuration differences between the development server and the Cloudflare Worker; resolving those differences was part of the implementation work.

5 Other project issues

Project management

The project did not follow a formal project-management method. I built the first React and Supabase version on 14 and 15 June 2025. The repository shows work on Cloudflare deployment beginning on 26 September 2026. I returned to the project on 6 October 2026, converted it to SvelteKit, and completed the main storage, upload, account, and community work from 6 to 7 October. The browser tests and final visual changes were added by 9 October. The dates below come from the Git history.

Work	Start	Finish	Result
Build the first React and Supabase version	14 June 2025	15 June 2025	Initial app with accounts, feed, communities, comments, playlists, and uploads.
Configure Cloudflare deployment	26 September 2026	26 September 2026	The React app was configured to run as a Cloudflare Worker.
Convert the app to SvelteKit	6 October 2026	6 October 2026	Pages and server routes moved into one SvelteKit project.
Move records and uploads to Cloudflare storage	7 October 2026	7 October 2026	D1 stores records; R2 stores video and image files.
Add remaining user features and security checks	7 October 2026	7 October 2026	Profiles, ownership, community membership, session, and input checks.
Add browser tests and final interface changes	7 October 2026	9 October 2026	Automated browser coverage and visual refinements.
Prepare the project report	12 October 2026	In progress	This report.

Table 2: Project work schedule.

The earlier version and the final version are part of the same project. The long pause between the initial prototype and the 2026 implementation is visible in the repository history.

Risk management

The biggest risk was a failed deployment when moving from Supabase to Cloudflare. The first deployment failed. Changes to the Cloudflare Worker configuration and the project configuration were needed before it worked. Environment variables also had to be configured, and the app had to use server-side rendering instead of being a static app.

The storage change created a second risk: losing or corrupting records during migration. There was no live Supabase data to move, so the new D1 schema could be created directly. The app stores the records in relational tables with foreign keys, and the migration is kept in the repository so the same schema can be applied to local and remote databases.

The session secret was also present in an earlier project configuration. I removed the fallback value and configured the production secret as a Worker secret. The app now returns an error if the session secret is missing, rather than using a public default. The production credentials are not stored in the repository.

Risk	Possible effect	Action taken
Incorrect Worker or SvelteKit configuration	The app would fail to build or would not serve pages correctly.	I adjusted the Worker configuration and checked the production deployment after changes.
Missing or exposed credentials	Sign-in could fail or sessions could be forged.	I moved production secrets out of the source configuration and made the app fail if a required session secret is missing.
Invalid or oversized media	A user could submit an unsupported file or consume excessive storage.	The server checks media type and size before writing to R2, and rate limits media requests.
A user changing another user's content	Users could lose control over their animations, comments, or playlists.	The server checks ownership before edits and deletions, and the smoke script checks a forbidden comment deletion.
Data loss during the	Existing records could be lost	No real user records existed to migrate. The

Risk	Possible effect	Action taken
move from Supabase	or corrupted.	new schema is versioned as SQL migrations.

Table 3: Project risks and actions.

These actions reduce the listed risks, but they do not prove that the system is free of security defects. The project has automated checks for selected cases and would need continued review as the application changes.

Configuration management

I tracked the source code and configuration with Git and GitHub. Each change is recorded in the repository history, which also shows when the main features and storage changes were made. The SQL schema is kept in a migration file. The Cloudflare bindings are declared in the Worker configuration. Local development values are kept in `.dev.vars`, which is not committed. This gives the local app and the deployed app separate credentials while keeping the structure of the application configuration under version control.

Testing

The repository includes browser end-to-end tests for the landing page, sign-in dialog keyboard behaviour, sign-up, animation upload, upvoting, commenting, and mobile community navigation. The report saved by the test runner records a passing browser test run on 9 October 2026. The API smoke script covers authentication, media uploads, playlists, communities, profiles, and ownership checks. I ran the API smoke script, and it passed.

The browser tests check a small number of complete user journeys. The sign-up test creates an account, uploads a sample video, opens the animation page, upvotes the animation, and posts a comment. The phone test uses a 375-pixel viewport, opens the communities page, and checks that the page does not scroll sideways. The dialog test checks Escape-key behaviour and keeps keyboard focus inside the open dialog.

The API smoke script checks expected responses and stored results. It covers sign-up and sign-in, Google sign-in redirects, password changes, profile updates, public profiles, media upload and video range requests, animation creation and listing, pagination and search, upvotes, comments, playlists, and community operations. It also checks that a second user cannot remove another user's comment and that invalid input is rejected.

These tests cover important user journeys and server behaviours, but they are not a complete measure of reliability, accessibility, or performance. I did not record formal user testing or a Lighthouse result for the final system, so this report does not make claims about those results.

Test	Type	Expected result	Recorded result
Guest landing page	Browser end-to-end	A visitor sees the project name and sign-up action.	Passed in the saved run dated 9 October 2026.
Sign-in dialog keyboard use	Browser end-to-end	Escape closes the dialog and Tab remains inside it while open.	Passed in the saved run dated 9 October 2026.
Sign-up, upload, upvote, and comment	Browser end-to-end	The user creates an account, publishes a video, then votes and comments.	Passed in the saved run dated 9 October 2026.
Community navigation on a phone viewport	Browser end-to-end	The community page opens at 375 pixels without sideways page scrolling.	Passed in the saved run dated 9 October 2026.
API smoke scenarios	API integration	Valid requests succeed and invalid or unauthorized requests return errors.	The script passed during project verification.

Table 4: Test results.

The browser suite uses a real browser against the local development server. The API smoke script sends HTTP requests to that server. These approaches test the application through its public interface, which is suitable for the user-visible requirements. The tests do not measure how the system behaves under many simultaneous users or with a large collection of media.

6 Conclusion

Evaluation of aims and objectives

The project achieved its main aim of giving animators a place to share work and for other users to find it. The feed shows recent or highly upvoted animations. Users can open an animation to comment on it, view its creator's public profile, and save animations in playlists. Users can also create communities and join or leave them.

Objective	Outcome
Provide a feed of animations in new or top order.	Achieved. The feed can be sorted and paged.
Let a signed-in user publish an animation with a title, description, and thumbnail.	Achieved. Video and thumbnail files are uploaded to R2 and the record is stored in D1.
Let users comment on and upvote animations.	Achieved. The animation page displays these interactions and their counts.
Let users create playlists and add animations.	Achieved. Playlist pages display the selected animations.
Let users create communities and join or leave them.	Achieved. Community membership and member counts are stored.
Let users find an animator's other work.	Achieved. Public profile pages list the creator's animations.

The outcome can be judged against the working application and its automated checks. It cannot be judged against a formal user survey because no such survey was carried out for the final version. The initial proposal also included features that were later removed or not implemented. The result is therefore narrower than the early idea, but the core sharing and discovery path works from account creation through upload to viewing and interaction.

The change from the early feature list is a limitation of the final product, but it also leaves a clearer system boundary. The completed work concentrates on publishing and finding animations, with account ownership and community membership around that activity. Features that need other roles

or services, such as employer accounts, applications, payment, or private messaging, remain outside the implementation. Adding them would require new requirements, data relationships, and tests rather than only adding links to the current pages.

The technical choices fit the delivered system: D1 holds related application records, R2 holds media files, and the Worker serves the SvelteKit application and its routes. The project has a repeatable database schema and automated checks for selected user journeys. The remaining work is to add missing product features only when they are required, improve media handling for larger or interrupted uploads, and collect user feedback before drawing conclusions about how well the platform serves animators in practice.

The project did not implement every feature considered during its early design. Community messages are not available. The finished system does not include job listings, applications, or paid content. These features would require further development.

Future development

Livestreaming would let animators show their creative process live. Premium content would let animators mark some content as premium so users, such as their fans, have to pay to view it. Community messaging could also be added so members can communicate with one another.

References

Cloudflare (2026a) ‘How Workers works’. Available at: https://developers.cloudflare.com/workers/reference/how-workers-works/ (Accessed: 14 October 2026).

Cloudflare (2026b) ‘Cloudflare D1’. Available at: https://developers.cloudflare.com/d1/ (Accessed: 14 October 2026).

Cloudflare (2026c) ‘Objects’. Available at: https://developers.cloudflare.com/r2/objects/ (Accessed: 14 October 2026).

Google (2026) ‘Using OAuth 2.0 for Web Server Applications’. Available at: https://developers.google.com/identity/protocols/oauth2/web-server (Accessed: 14 October 2026).

Svelte (2026) ‘Routing • SvelteKit Docs’. Available at: https://svelte.dev/docs/kit/routing (Accessed: 14 October 2026).

Appendices

A Test scripts

Table 5: Test scripts.

Test	Objective	Tasks	Expected result	Actual result
A1	Test account and upload functionality	Sign up with a new account, upload the supplied test video, open its page, upvote it, and add a comment.	The animation appears in the feed and detail page; the vote and comment appear.	Passed in the saved browser run on 9 October 2026.
A2	Test keyboard behaviour	Open the sign-in dialog, press Escape, reopen it, and press Tab repeatedly.	Escape closes the dialog; while open, focus stays inside it.	Passed in the saved browser run on 9 October 2026.
A3	Test mobile community navigation	Set the viewport to 375 × 667 pixels, open the home page, then open communities.	Communities can be opened and neither page scrolls sideways.	Passed in the saved browser run on 9 October 2026.
A4	Test API ownership checks	Sign in as two users, create a comment as the first user, and attempt to delete it as the second user.	The second user's request is refused; the author can delete the comment.	Passed in the API smoke script.
A5	Test media handling and profile validation	Upload a video and image, request a byte range from the video, and attempt an invalid avatar or website URL.	Valid media is stored and returned; the range response contains the requested bytes; invalid profile URLs are refused.	Passed in the API smoke script.

B User guide

1. Open the site and choose **sign up now** to create an account, or choose **sign in** if you already have one. You can also sign in with Google.
2. Browse the discover feed. Choose new or top order, or search for an animation. Open a video to view its description, comments, and creator.
3. To publish work, sign in and choose **add video**. Enter a title, add an optional description, select a video, and submit it. The app creates a thumbnail from the video.
4. Open your profile to edit your details, view your animations, and manage your playlists. You can create a playlist and add animations to it.
5. Open communities to create or join one. A community page shows its animations and lets members join or leave.
6. Sign out from the account menu when finished.

C Source code

The source code, SQL migrations, API smoke script, and browser tests are in the project repository: https://github.com/144126/animators-hive-hub.